

# Deployment Guide (Backend + Rule Engine + Swagger UI)

---

This guide explains how to **deploy the complete system** on any Linux server (Ubuntu recommended).  
Includes: backend, rule engine, ET module, and Swagger UI.

---

## 1. Prerequisites

---

Install required packages:

```
sudo apt update
sudo apt install -y git docker.io docker-compose python3 python3-pip nginx
```

Enable Docker:

```
sudo systemctl enable docker
sudo systemctl start docker
```

Clone your repo:

```
git clone https://github.com/<your-username>/smart-irrigation.git
cd smart-irrigation
```

---

## 2. Project Structure (Expected)

---

```
smart-irrigation/
├── api/
│   └── openapi.yaml
├── rule-engine/
│   ├── rule_engine.md
│   └── engine.py
├── deployment/
│   └── deployment_guide.md  <-- this file
├── backend/
│   └── app.py
├── docker-compose.yml
└── README.md
```

---

## 3. Environment Variables

---

Create a file:

```
nano .env
```

Add:

```
DB_URL=postgresql://admin:pass@db:5432/irrigation
WEATHER_API_KEY=YOUR_KEY      # API key for weather forecast integration
ET_MODE=hybrid
```

Save: **CTRL+O** → **ENTER** → **CTRL+X**

---

## 4. Python Backend Installation (Local / Manual)

---

Inside **backend/**:

```
cd backend
pip3 install -r requirements.txt
python3 app.py
```

Backend runs on: **http://localhost:8000**

---

## 5. Dockerizing the System

---

### 5.1 Create Dockerfile (backend)

Create:

```
nano backend/Dockerfile
```

Paste:

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
COPY . .
CMD ["python3", "app.py"]
```

Save.

---

## 5.2 docker-compose.yml

At project root:

```
nano docker-compose.yml
```

Paste:

```
version: "3.9"

services:

  backend:
    build: ./backend
    container_name: irrigation-backend
    ports:
      - "8000:8000"
    env_file: .env
    depends_on:
      - db

  db:
    image: postgres:15
    container_name: irrigation-db
    restart: always
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: irrigation
    ports:
      - "5432:5432"

  swagger:
    image: swaggerapi/swagger-ui
    container_name: irrigation-swagger
    environment:
      - SWAGGER_JSON=/openapi.yaml
    volumes:
      - ./api/openapi.yaml:/openapi.yaml
    ports:
      - "8081:8080"
```

Run all services:

```
docker-compose up --build -d
```

Check running containers:

```
docker ps
```

---

## 6. Reverse Proxy with Nginx

---

Install Nginx (already installed earlier):

```
sudo nano /etc/nginx/sites-available/irrigation
```

Paste:

```
server {  
    listen 80;  
    server_name your-server-ip;  
  
    location /api/ {  
        proxy_pass http://localhost:8000/;  
    }  
  
    location /docs/ {  
        proxy_pass http://localhost:8081/;  
    }  
}
```

Enable site:

```
sudo ln -s /etc/nginx/sites-available/irrigation /etc/nginx/sites-enabled/  
sudo systemctl restart nginx
```

---

## 7. Deployment Verification

---

Verify backend:

```
curl http://localhost/api/ping
```

Verify Swagger UI in browser:

```
http://your-server-ip/docs/
```

Verify DB:

```
docker exec -it irrigation-db psql -U admin -d irrigation
```

---

## 8. Logs & Monitoring

---

Check backend logs:

```
docker logs -f irrigation-backend
```

Check DB logs:

```
docker logs -f irrigation-db
```

---

## 9. Updating the Deployment

---

After making code changes:

```
git pull
docker-compose down
docker-compose up --build -d
```

---

## 10. Common Issues & Fixes

---

| Issue                | Cause            | Fix                     |
|----------------------|------------------|-------------------------|
| Backend not starting | Missing env vars | Check <code>.env</code> |

| Issue                     | Cause                    | Fix                     |
|---------------------------|--------------------------|-------------------------|
| Swagger not loading       | Wrong file mount         | Check volume path       |
| Database connection error | DB service slow to start | Add <b>depends_on</b>   |
| Port already in use       | Another app running      | Change ports in compose |

## 11. Production Hardening (Optional)

- Add HTTPS using Certbot
- Enable UFW firewall
- Use Docker secrets
- Enable auto-restart via Docker policies
- Add fail2ban for brute-force protection

## 12. Final Deployment Checklist

- ☐ Docker running
- ☐ Backend reachable
- ☐ DB initialized
- ☐ Rule engine connected
- ☐ Swagger UI visible
- ☐ Logs clean
- ☐ Nginx proxy working

## 13. Next Step

→ Review **troubleshooting\_guide.md** and **technicians\_faq.md** for operational guidance.